CS 6140 Project: Full Prep & Development Guide

Surrogate Aerodynamic Modeling — Transonic Airfoil Coefficient Prediction

Ashish Dasu

March 2026

Contents

1 Project Overview **2**
- 1.1 The core idea in one paragraph . . . 2
- 1.2 Why this matters for SpaceX specifically . . . 2
- 1.3 What we actually submitted . . . 2

2 Deep Q&A Prep **2**
- 2.1 Questions Elhamifar Will Likely Ask . . . 2
  - 2.1.1 "What do your features look like? Can you be more specific?" . . . 2
  - 2.1.2 "How do you handle multi-output regression?" . . . 2
  - 2.1.3 "What does 5-fold CV mean here in practice?" . . . 3
  - 2.1.4 "Why does polynomial regression fail at Mach 1?" . . . 3
  - 2.1.5 "Your dataset has only 1,362 points. Is that statistically meaningful?" . . . 3
  - 2.1.6 "Why not cross-validate across airfoils instead of randomly?" . . . 3
  - 2.1.7 "What does $R^2 = 0.996$ mean? Does it mean the problem is solved?" . . . 3
  - 2.1.8 "Why do you predict C_m if it's harder to get right?" . . . 3
- 2.2 Questions a Classmate Might Ask . . . 4
  - 2.2.1 "What's a drag polar?" . . . 4
  - 2.2.2 "What are these airfoils? What does NACA0012 mean?" . . . 4
  - 2.2.3 "What's the difference between this and just running CFD?" . . . 4
- 2.3 Dangerous Questions — Know These Cold . . . 4
  - 2.3.1 "What does rhoCentralFoam solve?" . . . 4
  - 2.3.2 "Why is your split 60/20/20 and not 80/10/10?" . . . 4
  - 2.3.3 "What's your null hypothesis baseline?" . . . 4

3 Technical Background You Should Know **4**
- 3.1 Aerodynamics . . . 4
- 3.2 ML Concepts You'll Need to Explain . . . 5

4 Development Plan **5**
- 4.1 Phase 0: Setup (Day 1) . . . 5
- 4.2 Phase 1: EDA (Day 1–2) . . . 6
- 4.3 Phase 2: Data Pipeline (Day 2) . . . 6
- 4.4 Phase 3: Polynomial Regression Baseline (Day 2–3) . . . 7
- 4.5 Phase 4: Random Forest (Day 3) . . . 8

4.6 Phase 5: XGBoost (Day 3–4) 9
4.7 Phase 6: Deep Neural Network (Days 4–6) 9
4.8 Phase 7: Evaluation & Figures (Day 6–7) 11
4.9 Phase 8: Inference Latency Benchmark (Day 7) 13
4.10 Phase 9: Results Summary Table 13

5 Pitfalls to Avoid **14**

6 Timeline **14**

7 GitHub Setup **15**

Project Overview

The core idea in one paragraph

Predicting aerodynamic coefficients — lift (C_l), drag (C_d), pitching moment (C_m) — normally requires running a CFD simulation for each new set of flight conditions and airfoil geometry. That takes 7 minutes per run on 128 CPU cores. For anything that needs thousands of evaluations (trajectory optimization, GNC tuning, design exploration), that's a hard wall. We're training regression models on pre-computed CFD data so we can query a model in milliseconds instead. The transonic regime (Mach 0.3–0.8) is the focus because that's where aerodynamics gets genuinely nonlinear — shocks form, drag spikes discontinuously — and where the naive interpolation approach breaks down.

Why this matters for SpaceX specifically

Stage separation on Falcon 9 happens around Mach 2, but the grid fins that control the first stage during reentry operate through the transonic regime. Starship's reentry is entirely transonic/subsonic. GNC teams need accurate aerodynamic tables to compute trajectories — and they need them fast. An ML surrogate that replaces lookup tables with a learned function is exactly what gets used in practice.

What we actually submitted

Dataset: TransonicSurrogate — 1,362 CFD simulations, 8 airfoils, Mach 0.3–0.8, AoA −4 to +8, split 60/20/20. Freely available at `https://github.com/Mohamedelrefaie/TransonicSurrogate`.

Models: Polynomial regression (baseline), Random Forest (sklearn), Gradient Boosting (XGBoost), Deep Neural Network (PyTorch MLP, multi-output).

Evaluation: RMSE and R^2 per coefficient on test set, drag polar comparison vs. CFD, inference latency.

Deep Q&A Prep

Questions Elhamifar Will Likely Ask

"What do your features look like? Can you be more specific?"

The dataset represents each airfoil via a set of geometric parameters (thickness, camber, and related profile descriptors) combined numerically into a single feature vector alongside Mach and AoA. The exact column names are in `Input_Data.csv` which I'll confirm on download. The key design decision in the paper is that they parameterize geometry numerically rather than using a one-hot airfoil ID — this means the model sees geometry as a continuous input and can in principle generalize to interpolated shapes, not just the 8 it trained on.

"How do you handle multi-output regression?"

For sklearn models (Random Forest, polynomial): use `MultiOutputRegressor` wrapper or native multi-output support. RF natively supports it. For XGBoost: train one model per output, which lets you tune each independently. For the DNN: shared trunk with three separate output heads,

trained jointly with MSE loss summed across all three outputs. The joint training means the trunk learns a shared representation for all three coefficients simultaneously.

"What does 5-fold CV mean here in practice?"

The 1,362 samples are split into 5 folds of 272 samples each. For each fold, we train on 4 folds (1,090 samples) and validate on the remaining fold. We repeat this 5 times so every sample appears in the validation set exactly once. This gives a more robust estimate of generalization error than a single train/val split, especially important here given the small dataset. For hyperparameter tuning, we pick the hyperparameter values that minimize mean validation RMSE across folds.

"Why does polynomial regression fail at Mach 1?"

Drag divergence creates a sharp near-discontinuity in C_d around Mach 0.8–1.0 — it rises steeply and nonlinearly as shocks form. A degree-2 or degree-3 polynomial is a smooth global function and will try to "average over" this discontinuity, fitting it poorly everywhere near the transition. Tree-based models can capture it via decision boundaries. The DNN can learn it via composition of nonlinear activations.

"Your dataset has only 1,362 points. Is that statistically meaningful?"

For this feature dimensionality it is. We have roughly 10 features and the task is regression, not high-dimensional generative modeling. The bias-variance tradeoff favors simpler models at this scale — which is exactly why polynomial regression and RF might perform comparably to the DNN. The original paper achieves $R^2 = 99.6\%$ on this data, suggesting it's learnable. The interesting question is whether a DNN overfits at this scale, which is something we'll observe directly.

"Why not cross-validate across airfoils instead of randomly?"

That would be a stricter evaluation — leave-one-airfoil-out CV tests whether the model generalizes to geometries it's never seen. Random 5-fold CV mixes data from all 8 airfoils into every fold, so the model has seen every airfoil during training. We chose random CV because it matches the original paper's protocol and fits the rubric requirement. Cross-airfoil generalization is something we mention as an analysis (compare predictions on each airfoil), not a training-time split.

"What does $R^2 = 0.996$ mean? Does it mean the problem is solved?"

It means the model explains 99.6% of variance in the target on the test set. That sounds like a solved problem, but there are two important caveats. First, that's on the same airfoil distribution as training — generalization to new airfoil shapes could be much worse. Second, for GNC use you care about absolute prediction error, not just variance explained. A model with high R^2 can still have systematic errors at specific Mach regimes (e.g., near drag divergence) that matter operationally.

"Why do you predict C_m if it's harder to get right?"

C_m determines pitch stability — whether the vehicle pitches nose-up or nose-down in response to a perturbation. For trajectory simulation and GNC, you need all three coefficients. A surrogate that only predicts C_l and C_d is incomplete. C_m is also the hardest to predict (most sensitive to small geometric changes), so it's the most scientifically interesting target.

Questions a Classmate Might Ask

"What's a drag polar?"

A drag polar is a plot of C_l vs. C_d — it shows the tradeoff between lift and drag across angle of attack. Every aerodynamicist uses drag polars to characterize an airfoil's efficiency. We use it as a sanity check: even if RMSE is low, if the predicted drag polar looks nothing like the CFD drag polar, the model has learned something physically wrong.

"What are these airfoils? What does NACA0012 mean?"

NACA airfoils are standardized profiles defined by a 4 or 5-digit code. NACA0012: first two digits (00) mean zero camber (symmetric), last two (12) mean 12% max thickness-to-chord ratio. NACA2412: 2% max camber at 40% chord, 12% thickness. RAE2822 is a British transonic research airfoil used as a benchmark in CFD validation since the 1970s.

"What's the difference between this and just running CFD?"

Speed. CFD at 7 minutes per query means you can run 200 queries in a day on a cluster. The trained model runs in microseconds, so you can run millions of queries for trajectory optimization or Monte Carlo uncertainty analysis. The tradeoff is accuracy — the model is an approximation, not the ground truth.

Dangerous Questions — Know These Cold

"What does rhoCentralFoam solve?"

It solves the compressible Euler equations — conservation of mass, momentum, and energy for inviscid (no viscosity) compressible flow. It uses the Kurganov-Tadmor central scheme, which handles shocks well. It is **not** RANS (Reynolds-Averaged Navier-Stokes) and does **not** use the Spalart-Allmaras turbulence model. Inviscid is appropriate here because pressure forces dominate in transonic aerodynamic coefficient prediction.

"Why is your split 60/20/20 and not 80/10/10?"

That's what the original paper uses. With 1,362 samples, 60% training = 817 samples, 20% validation = 272 samples, 20% test = 273 samples. The larger validation and test sets give more reliable estimates of model performance at this small sample size.

"What's your null hypothesis baseline?"

Predicting the mean of the training targets for every test point. This gives $R^2 = 0$ by definition. Polynomial regression should beat this easily. The question is by how much, and whether the nonlinear models provide meaningful improvement over polynomial near drag divergence.

Technical Background You Should Know

Aerodynamics

- C_l **(lift coefficient):** Non-dimensional lift force. $C_l = L/(0.5\rho V^2 A)$. Increases with AoA up to stall.

- C_d **(drag coefficient):** Non-dimensional drag force. Has two components: pressure drag (dominant at transonic) and viscous/friction drag.
- C_m **(pitching moment coefficient):** Non-dimensional moment about the aerodynamic center. Negative = nose-down (stable). Critical for GNC.
- **Mach number:** Flow velocity divided by speed of sound. Mach 1 = sonic. Transonic = 0.3–0.8 (local supersonic regions form).
- **Drag divergence:** The Mach number at which C_d rises sharply due to shock formation. Typically Mach 0.7–0.85 depending on airfoil.
- **Angle of attack (AoA):** Angle between the airfoil chord line and the oncoming flow. Increasing AoA increases C_l (up to stall) and C_d.

ML Concepts You'll Need to Explain

- **Multi-output regression:** Single model predicts multiple targets simultaneously. Contrast with training separate models per target.
- **Basis function expansion:** Polynomial regression lifts input features to higher-dimensional space. For degree 2 with p features: $O(p^2)$ features added.
- **Ensemble methods:** RF averages many decorrelated trees. XGBoost builds trees sequentially to correct residuals.
- **Surrogate model:** A fast approximation of an expensive function. In aerospace, ML surrogates replace CFD or wind tunnel lookup tables.
- **RMSE vs. R^2:** RMSE is in the units of the target (interpretable). R^2 is unitless fraction of variance explained. Use both.

Development Plan

Phase 0: Setup (Day 1)

1. Clone the repo: `git clone https://github.com/Mohamedelrefaie/TransonicSurrogate`
2. Open `Input_Data.csv` in pandas, print `df.head()`, `df.columns`, `df.describe()`, `df.shape`
3. Confirm: exact column names, number of features, number of targets, any missing values
4. Update the proposal feature description with the real column names once confirmed
5. Set up repo structure:

```
cs6140-project/
  data/
    Input_Data.csv
  notebooks/
    01_eda.ipynb
    02_baseline.ipynb
    03_random_forest.ipynb
    04_xgboost.ipynb
    05_dnn.ipynb
    06_evaluation.ipynb
  src/
    data.py         # loading, splitting, normalization
    models.py       # model definitions
    evaluate.py     # metrics, drag polar plots
    train_dnn.py    # PyTorch training loop
  results/
    figures/
```

```
    tables/
  README.md
```

Phase 1: EDA (Day 1–2)

Before touching any model, understand the data completely. This is what separates a good project from a bad one.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data/Input_Data.csv')
print(df.shape)        # confirm 1362 rows
print(df.columns)      # confirm exact column names
print(df.describe())   # check ranges, no NaNs
print(df.dtypes)       # confirm all numeric

# Check for missing values
print(df.isnull().sum())

# Distribution of each feature
df.hist(bins=30, figsize=(15, 10))
plt.tight_layout()
plt.savefig('results/figures/feature_distributions.png')

# Correlation matrix
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.savefig('results/figures/correlation_matrix.png')

# Key relationship: Cd vs Mach (should show drag divergence)
plt.scatter(df['Mach'], df['Cd'], alpha=0.3, c=df['AoA'])
plt.xlabel('Mach')
plt.ylabel('Cd')
plt.colorbar(label='AoA')
plt.savefig('results/figures/cd_vs_mach.png')
```

What to look for in EDA:

- Does C_d show a visible spike near Mach 0.8? (drag divergence — this is the nonlinear signal your models need to capture)
- Are any features highly correlated? (could affect polynomial regression stability)
- Is the data roughly uniformly sampled across Mach and AoA? (if not, weighting may matter)
- Are there outliers? (especially in C_m which is noisier)

Phase 2: Data Pipeline (Day 2)

Build this once and reuse it everywhere. Sloppy data handling is the most common source of bugs.

```python
# src/data.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler

def load_data(path='data/Input_Data.csv'):
    df = pd.read_csv(path)
    # UPDATE these after confirming column names from CSV
    feature_cols = [col for col in df.columns
                    if col not in ['Cl', 'Cd', 'Cm']]
    target_cols  = ['Cl', 'Cd', 'Cm']
    X = df[feature_cols].values
    y = df[target_cols].values
    return X, y, feature_cols, target_cols

def split_data(X, y, val_size=0.2, test_size=0.2, seed=42):
    # First split off test (20%)
    X_tmp, X_test, y_tmp, y_test = train_test_split(
        X, y, test_size=test_size, random_state=seed)
    # Then split val from remaining (20% of original = 25% of tmp)
    val_frac = val_size / (1 - test_size)
    X_train, X_val, y_train, y_val = train_test_split(
        X_tmp, y_tmp, test_size=val_frac, random_state=seed)
    return X_train, X_val, X_test, y_train, y_val, y_test

def normalize(X_train, X_val, X_test, y_train, y_val, y_test):
    # Fit scalers on training data ONLY -- never on val or test
    X_scaler = StandardScaler().fit(X_train)
    # Separate scaler per output coefficient
    y_scaler = StandardScaler().fit(y_train)

    X_train = X_scaler.transform(X_train)
    X_val   = X_scaler.transform(X_val)
    X_test  = X_scaler.transform(X_test)
    y_train = y_scaler.transform(y_train)
    y_val   = y_scaler.transform(y_val)
    y_test  = y_scaler.transform(y_test)
    return (X_train, X_val, X_test,
            y_train, y_val, y_test,
            X_scaler, y_scaler)
```

Critical note: Always fit the scaler on training data only. Fitting on the full dataset leaks test set statistics into your normalization and inflates your results. This is a common mistake that Elhamifar will catch.

Phase 3: Polynomial Regression Baseline (Day 2–3)

```
# notebooks/02_baseline.ipynb
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, KFold
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
results = {}
for degree in [2, 3]:
    pipe = Pipeline([
        ('poly',  PolynomialFeatures(degree=degree,
                                     include_bias=False)),
        ('ridge', Ridge(alpha=1.0))   # L2 reg prevents explosion
    ])
    # CV on training data only
    scores = cross_val_score(pipe, X_train, y_train,
                             cv=kf, scoring='r2')
    results[degree] = scores.mean()
    print(f"Degree␣{degree}:␣CV␣R2␣=␣{scores.mean():.4f}␣"
          f"(+/-␣{scores.std():.4f})")

# Fit best degree on full training set
best_degree = max(results, key=results.get)
pipe_final = Pipeline([
    ('poly',  PolynomialFeatures(degree=best_degree,
                                 include_bias=False)),
    ('ridge', Ridge(alpha=1.0))
])
pipe_final.fit(X_train, y_train)
y_pred = pipe_final.predict(X_test)

# Evaluate per-coefficient
for i, name in enumerate(['Cl', 'Cd', 'Cm']):
    r2   = r2_score(y_test[:, i], y_pred[:, i])
    rmse = mean_squared_error(y_test[:, i],
                              y_pred[:, i],
                              squared=False)
    print(f"{name}:␣R2={r2:.4f},␣RMSE={rmse:.6f}")
```

Phase 4: Random Forest (Day 3)

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth':    [5, 10, None],  # None = unlimited
    'min_samples_split': [2, 5],
}

rf = RandomForestRegressor(random_state=42, n_jobs=-1)
search = GridSearchCV(rf, param_grid, cv=5,
                      scoring='r2', n_jobs=-1, verbose=1)
search.fit(X_train, y_train)

print("Best␣params:", search.best_params_)
print("Best␣CV␣R2:", search.best_score_)

rf_best = search.best_estimator_
```

```
y_pred_rf = rf_best.predict(X_test)

# Feature importance (averaged across all outputs)
importances = rf_best.feature_importances_
for feat, imp in sorted(zip(feature_cols, importances),
                        key=lambda x: -x[1]):
    print(f"{feat}: {imp:.4f}")
```

Phase 5: XGBoost (Day 3–4)

XGBoost doesn't natively support multi-output, so train one model per target.

```
import xgboost as xgb
from sklearn.model_selection import GridSearchCV

param_grid = {
    'learning_rate':  [0.01, 0.05, 0.1],
    'max_depth':      [3, 5, 7],
    'n_estimators':   [100, 300, 500],
    'subsample':      [0.8, 1.0],
}

xgb_models = {}
for i, name in enumerate(['Cl', 'Cd', 'Cm']):
    print(f"\nTraining XGBoost for {name}...")
    model = xgb.XGBRegressor(random_state=42, n_jobs=-1)
    search = GridSearchCV(model, param_grid, cv=5,
                          scoring='r2', n_jobs=-1)
    search.fit(X_train, y_train[:, i])
    xgb_models[name] = search.best_estimator_
    print(f"  Best CV R2: {search.best_score_:.4f}")
    print(f"  Best params: {search.best_params_}")

# Predict all three
y_pred_xgb = np.column_stack([
    xgb_models[name].predict(X_test)
    for name in ['Cl', 'Cd', 'Cm']
])
```

Phase 6: Deep Neural Network (Days 4–6)

This is the most involved part. Build it carefully.

```
# src/train_dnn.py
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

class AeroMLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, n_outputs=3,
                 dropout_p=0.1):
        super().__init__()
        layers = []
```

```
        prev = input_dim
        for h in hidden_dims:
            layers += [
                nn.Linear(prev, h),
                nn.BatchNorm1d(h),
                nn.ReLU(),
                nn.Dropout(dropout_p)
            ]
            prev = h
        self.trunk = nn.Sequential(*layers)
        # Three separate output heads
        self.head_cl = nn.Linear(prev, 1)
        self.head_cd = nn.Linear(prev, 1)
        self.head_cm = nn.Linear(prev, 1)

    def forward(self, x):
        z = self.trunk(x)
        return torch.cat([
            self.head_cl(z),
            self.head_cd(z),
            self.head_cm(z)
        ], dim=1)

def train_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(xb)
    return total_loss / len(loader.dataset)

def val_epoch(model, loader, criterion, device):
    model.eval()
    total_loss = 0
    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            pred = model(xb)
            total_loss += criterion(pred, yb).item() * len(xb)
    return total_loss / len(loader.dataset)

def train_model(X_train, y_train, X_val, y_val,
                hidden_dims, dropout_p=0.1,
                lr=1e-3, epochs=300, batch_size=64,
                patience=30, device='cuda'):
    device = torch.device(device if torch.cuda.is_available()
                          else 'cpu')
    X_tr = torch.FloatTensor(X_train)
    y_tr = torch.FloatTensor(y_train)
```

```
    X_v  = torch.FloatTensor(X_val)
    y_v  = torch.FloatTensor(y_val)

    train_loader = DataLoader(
        TensorDataset(X_tr, y_tr),
        batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(
        TensorDataset(X_v, y_v),
        batch_size=batch_size)

    model = AeroMLP(X_train.shape[1], hidden_dims,
                    dropout_p=dropout_p).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, patience=10, factor=0.5)
    criterion = nn.MSELoss()

    best_val, best_state, wait = float('inf'), None, 0
    for epoch in range(epochs):
        train_loss = train_epoch(
            model, train_loader, optimizer, criterion, device)
        val_loss = val_epoch(
            model, val_loader, criterion, device)
        scheduler.step(val_loss)

        if val_loss < best_val:
            best_val = val_loss
            best_state = {k: v.clone()
                          for k, v in model.state_dict().items()}
            wait = 0
        else:
            wait += 1
            if wait >= patience:
                print(f"Early␣stop␣at␣epoch␣{epoch}")
                break

    model.load_state_dict(best_state)
    return model
```

Architecture search: Tune these hyperparameters, tracking validation loss for each:

Hyperparameter	Values to try
Hidden dims	[128,128], [256,256], [256,128,64], [256,256,128], [512,256,128]
Dropout	0.1, 0.3
Learning rate	1e-3, 5e-4
Batch size	32, 64

Start with [256,128] as a reasonable default and verify it trains stably before doing the full search.

Phase 7: Evaluation & Figures (Day 6–7)

This is what the report grades you on. Every result should be a clean figure or table.

```python
# src/evaluate.py
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score, mean_squared_error

def evaluate_all(models_dict, X_test, y_test, y_scaler,
                 target_names=['Cl', 'Cd', 'Cm']):
    """
    models_dict: {'Poly': y_pred_poly, 'RF': y_pred_rf, ...}
    Returns a DataFrame of RMSE and R2 per model per target
    """
    results = []
    for model_name, y_pred_norm in models_dict.items():
        # Inverse transform to original units
        y_pred = y_scaler.inverse_transform(y_pred_norm)
        y_true = y_scaler.inverse_transform(y_test)
        for i, tgt in enumerate(target_names):
            r2   = r2_score(y_true[:,i], y_pred[:,i])
            rmse = mean_squared_error(
                y_true[:,i], y_pred[:,i], squared=False)
            results.append({
                'Model': model_name,
                'Target': tgt,
                'R2': r2,
                'RMSE': rmse
            })
    return pd.DataFrame(results)

def plot_drag_polar(y_true, y_pred_dict,
                    save_path='results/figures/drag_polar.png'):
    """
    Compare Cl vs Cd curves across models.
    Sort by Cl for a clean curve.
    """
    fig, axes = plt.subplots(1, len(y_pred_dict)+1,
                             figsize=(5*(len(y_pred_dict)+1), 5))
    # Ground truth
    idx = np.argsort(y_true[:,0])
    axes[0].plot(y_true[idx,1], y_true[idx,0], 'k-', lw=2)
    axes[0].set_title('CFD␣Ground␣Truth')
    axes[0].set_xlabel('Cd')
    axes[0].set_ylabel('Cl')

    for ax, (name, y_pred) in zip(axes[1:], y_pred_dict.items()):
        idx = np.argsort(y_pred[:,0])
        ax.plot(y_true[idx,1],  y_true[idx,0],  'k--',
                label='CFD', alpha=0.5)
        ax.plot(y_pred[idx,1],  y_pred[idx,0],  'r-',
                label=name, lw=2)
        ax.set_title(name)
        ax.set_xlabel('Cd')
        ax.legend()
    plt.tight_layout()
```

```
    plt.savefig(save_path, dpi=150)

def plot_parity(y_true, y_pred, model_name,
                target_names=['Cl','Cd','Cm']):
    """Predicted vs actual scatter for each coefficient."""
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    for i, (ax, name) in enumerate(zip(axes, target_names)):
        ax.scatter(y_true[:,i], y_pred[:,i], alpha=0.3, s=10)
        lims = [min(y_true[:,i].min(), y_pred[:,i].min()),
                max(y_true[:,i].max(), y_pred[:,i].max())]
        ax.plot(lims, lims, 'r--', lw=1)
        r2 = r2_score(y_true[:,i], y_pred[:,i])
        ax.set_title(f'{model_name}␣  ␣{name}␣(R  ={r2:.4f})')
        ax.set_xlabel('CFD')
        ax.set_ylabel('Predicted')
    plt.tight_layout()
    plt.savefig(
        f'results/figures/parity_{model_name.lower()}.png',
        dpi=150)
```

Phase 8: Inference Latency Benchmark (Day 7)

Required since we claim millisecond inference. Measure it properly.

```
import time

def benchmark_latency(model, X_test, n_runs=100,
                      model_type='sklearn'):
    # Single-sample latency (real use case for GNC)
    single = X_test[0:1]
    times = []
    for _ in range(n_runs):
        t0 = time.perf_counter()
        if model_type == 'sklearn':
            _ = model.predict(single)
        elif model_type == 'torch':
            with torch.no_grad():
                _ = model(torch.FloatTensor(single))
        times.append(time.perf_counter() - t0)

    print(f"Single-sample:␣{np.mean(times)*1000:.3f}␣ms␣"
          f"(+/-␣{np.std(times)*1000:.3f}␣ms)")
    print(f"Speedup␣vs␣7␣min␣CFD:␣"
          f"{7*60␣/␣np.mean(times):.0f}x")
```

Phase 9: Results Summary Table

This goes directly into your report Section 4. Build it programmatically.

Model	Target	R^2	RMSE	Train time	Inference
Polynomial (deg 2)	C_l	?	?		
	C_d	?	?		
	C_m	?	?		
Random Forest	C_l	?	?		
	C_d	?	?		
	C_m	?	?		
XGBoost	C_l	?	?		
	C_d	?	?		
	C_m	?	?		
DNN (PyTorch)	C_l	?	?		
	C_d	?	?		
	C_m	?	?		

Pitfalls to Avoid

1. **Data leakage in normalization:** Always fit StandardScaler on training data only. Never on the full dataset.
2. **Inverse transforming predictions:** All metrics should be reported in the original (unnormalized) scale. Remember to call `y_scaler.inverse_transform()` before computing RMSE/R^2.
3. **XGBoost multi-output:** Train one model per target, not one model with the full y matrix (XGBoost doesn't support it natively).
4. **DNN overfitting:** With 817 training samples, a large DNN will overfit. Watch the train/val loss gap. If val loss plateaus while train loss keeps dropping, add dropout or reduce capacity.
5. **Reporting test metrics prematurely:** Do all hyperparameter tuning on train/val only. Look at test set once, at the very end.
6. **Not reproducing results:** Set `random_state=42` everywhere. Save model checkpoints. Your GitHub must reproduce your exact numbers.

Timeline

Phase	Task	Target
0	Setup, download CSV, confirm columns	Day 1
1	EDA, distribution plots, Cd vs Mach	Days 1–2
2	Data pipeline (load, split, normalize)	Day 2
3	Polynomial regression + CV	Day 2–3
4	Random Forest + grid search	Day 3
5	XGBoost per-output + grid search	Days 3–4
6	DNN architecture search + early stopping	Days 4–6
7	Evaluation figures (parity, drag polar)	Days 6–7
8	Latency benchmark	Day 7
9	Report writing	Days 7–10

GitHub Setup

The grader will run your code. Make this painless for them.

```
# requirements.txt
numpy>=1.24
pandas>=2.0
scikit-learn>=1.3
xgboost>=2.0
torch>=2.0
matplotlib>=3.7
seaborn>=0.12
jupyter>=1.0
```

README must include:

1. Dataset download instructions (just clone the TransonicSurrogate repo)
2. `pip install -r requirements.txt`
3. How to run each notebook in order
4. What the expected outputs are